Bridging the "Evaluation Trust" Gap: A Unified Framework for Quantifying Semantic Robustness and Test Adequacy in Code LLMs

MSc Research Proposal, Nouha Karoui

April 24, 2026

Executive summary

This proposal describes a unified, reproducible research programme focused on Python that combines code-level mutation testing and prompt-level metamorphic (paraphrase) testing to evaluate and improve execution-based code benchmarks (e.g., HumanEval, MBPP). The project builds an open toolchain, Meta-Real-Eval, that:

1. Pre-filters equivalent mutants via automated differential fuzzing (Stage 0) before all kill-rate analysis, ensuring comparisons are not confounded by semantically identical programs.
2. Characterises LLM-specific fault profiles (RQ1a) and separately measures how adequate benchmark test suites are at detecting those faults (RQ1b).
3. Produces controlled paraphrases and metamorphic prompts to quantify ranking instability (RQ2), and re-applies this analysis at each benchmark degradation level to test whether oracle weakness and prompt sensitivity compound each other (RQ4 interaction).
4. Implements an automated validation framework using formally defined implementation divergence and the hybrid SBC (Semantic-BLEU-Completeness) metric to detect false positives without human oracles (RQ3).
5. Demonstrates remediation by using Consistency Assertions: oracle-free cross-variant checks generated (RQ3), to restore evaluation trust even when benchmark test suites are intentionally weakened (RQ4).

1 Research problem

Execution-based benchmarks for code generation are the primary standard for claiming progress in Code LLMs. However, these benchmarks suffer from a profound "Evaluation Trust" gap driven by three interacting pathologies:

- **Oracle Incompleteness:** Test suites are often too small or lack the branch coverage to detect subtle logic errors.
- **Prompt Sensitivity:** Minor paraphrases can flip model rankings, suggesting benchmarks measure prompt-alignment rather than reasoning.
- **Fault Mismatch:** Traditional mutation operators may not reflect the "sneaky" semantic faults that LLMs actually produce.

This thesis treats benchmarks as the object of study rather than a static metric. It investigates whether combining code mutation and metamorphic prompt testing can:

1. (RQ1a) Characterise the LLM-specific fault profile and determine whether LLM faults form a structurally distinct class not captured by traditional operators.

2. (RQ1b) Diagnose benchmark adequacy specifically, whether existing test suites can detect the fault types identified in RQ1a.

3. (RQ2) Predict ranking instability through semantic invariance testing, and quantify whether this instability compounds with oracle weakness.

4. (RQ3) Detect false positives automatically using implementation divergence and the SBC metric, without relying on human oracles.

5. (RQ4) Remediate evaluation trust loss through Consistency Assertions, and test whether MT-augmented benchmarks stabilise rankings under both test removal and prompt variation.

2 State of the art and gaps

2.1 Representative prior work

- **Benchmark adequacy studies.** Empirical analyses show that popular benchmarks have limited branch/mutation adequacy and that augmentations can improve rigor. [6].
- **Metamorphic / prompt testing.** Paraphrase/metamorphic prompt testing has been proposed to detect inconsistent or incorrect LLM outputs; systematic reviews summarize transformations and techniques [3] [7].
- **LLM-based mutation generation.** LLMs can propose realistic mutants that traditional mutators miss [10].
- **Mutation-based LLM evaluation.** Mutation-based consistency testing and MCT (Mutation-based Consistency Testing) methods evaluate LLMs' code understanding by injecting mutants and checking model responses [5]).
- **Reverse-generation metrics.** Reverse generation (requirements inferred from code) and hybrid SBC (Semantic-BLEU-Completeness) metrics provide complementary signals to test execution [8].

2.2 Identified Gaps

1. Most work studies either code mutation or prompt paraphrase in isolation; a unified pipeline combining both to evaluate *benchmarks* is missing.
2. It is unclear whether mutating canonical solutions vs. mutating LLM outputs yields different diagnostic signals.
3. There is limited evidence linking benchmark adequacy signals (mutation score, coverage) to *ranking stability* under prompt and test perturbations.
4. Practical remediation loops (mutation-guided test augmentation + metamorphic checks) have not been systematically evaluated for their effect on ranking stability.
5. No prior work has examined whether oracle incompleteness and prompt sensitivity compound each other (i.e., whether a weaker test suite amplifies ranking instability under paraphrase). This interaction is a central novel contribution of this thesis.

3 Planned Research

The primary objective of this research is to treat existing execution-based benchmarks (e.g., HumanEval, MBPP) as objects of study to diagnose systemic fragilities, specifically oracle incompleteness and prompt sensitivity. By shifting the focus from model performance to benchmark adequacy, this work aims to quantify the degree to which current leaderboards reflect genuine algorithmic reasoning versus data memorization or prompt alignment.

3.1 Experimental Design: Tiered Approach

To manage computational overhead and model non-determinism, the study employs a two-tier empirical design:

1. **Tier 1 (Pilot):** .Intensive, high-repetition experiments on **HumanEval** to establish baseline configurations. This phase uses n=10 repetitions per prompt to calculate stable Pass@k distributions and identify "worst-case" tasks where models exhibit the highest variance

2. **Tier 2 (Generalization):** The optimized configuration will be applied to **RealClassEval** [9], evaluating model behavior at the project-level. This tier tests whether semantic robustness insights from standalone functions generalize to complex object-oriented dependencies

3.2 The Meta-Real-Eval Toolchain

The research utilizes the Meta-Real-Eval toolchain, an integrated Python-based pipeline designed to stress-test the interaction between models and test suites. The toolchain consists of four modular engines:

- **Stage 0 Prerequisite - Differential Fuzzing Module (Equivalence Filter:** This module runs before any kill-rate computation in RQ1. For each generated mutant m and original program p:
  - Execute m and p on N = 500 random inputs sampled from the task's input domain.
  - If no output divergence is observed across all N inputs, flag m as likely-equivalent and exclude from kill-rate calculations.
  - Apply AST-level subsumption as a secondary filter for obvious structural equivalence.
- **AST-Level Code Mutator:** Using **Cosmic-Ray** [1] the engine seeds traditional syntactic faults (e.g., AOR, ROR, SDL) into both canonical and LLM-generated code. **Kill Ratios** are computed only on non-equivalent mutants (post Stage 0 filtering).
- **Syntax-Aware Prompt Mutator:** Decomposes prompt templates into syntax trees to generate 100 semantically invariant paraphrases per task. Applies Metamorphic Relations (fact reordering, persona-shifting, syntactic expansion) to measure the semantic invariance of model reasoning across prompt variants. [4]
- **Automated Test/Requirement Orchestrator:** Integrates **Pynguin** [2] for search-based test generation and a Requirement Reconstructor. The latter asks the LLM to reverse-generate requirements from its code to calculate the **SBC Score (Semantic, BLEU, Completeness)**, providing a validation signal independent of fixed oracles [3] [8].

- **MT Augmentation Engine:** Converts divergence signals from RQ3 into Consistency Assertions oracle-free cross-variant checks, requiring that solutions generated from k paraphrased prompts produce identical outputs on shared random inputs. These assertions constitute the "MT-augmented" test suite used in RQ4, defined as: remaining unit tests + Consistency Assertions.

3.3 Statistical Rigor and Compute Mitigation

Each prompt variant is sampled across $n = 10$ iterations to account for sampling stochasticity. The following statistical procedures are applied across the RQs:

- **Kill-rate comparisons (RQ1a, RQ1b):** Wilcoxon signed-rank test (paired by task) with Cliff's δ as the effect-size measure.
- **Ranking instability (RQ2, RQ4):** Kendall's τ_b with bootstrap 95% confidence intervals ($B = 1,000$ resamples) per model family.
- **Rank recovery (RQ4):** Spearman's ρ and Kendall's τ_b of degraded/augmented rankings versus the full-suite baseline.
- **Interaction experiment (RQ4 $\times$ RQ2):** The RQ2 τ_b analysis is re-run at each degradation level (0%, 20%, 50%, 80%). The resulting τ_b values are regressed against degradation percentage; monotonicity is tested with Page's L trend test. This quantifies whether prompt sensitivity and oracle weakness compound each other.
- **False-positive detection (RQ3):** ROC-AUC of the divergence-rate classifier against oracle pass/fail labels.

To mitigate compute costs, stratified sampling is employed. Exhaustive mutation analysis is reserved for tasks with high cyclomatic complexity or significant prompt sensitivity identified in the Tier 1 pilot; the remainder undergoes lighter metamorphic consistency checks only.

4 Research Questions and Hypotheses

Table 1: Research Questions, Hypotheses, and Experimental Methodology

Research Question	Hypotheses (H1 / H0)	Design & Data	Method & Analysis
RQ1a: Fault Characterisation. M faults form a distinct class poorly captured by traditional operators?	**H1:** aults (wrong logic, hallucinated APIs) survive AOR/ROR/SDL at significantly higher rates. **H0:** rates are indistinguishable across mutation sources.	Two mutant populations per task: (a) Cosmic-Ray (AOR, ROR, SDL); (b) LLMorpheus semantic mutants. Applied to HumanEval (Tier 1) and RealClassEval (Tier 2). *Equivalent mutants pre-filtered via Stage 0 differential fuzzing.*	Wilcoxon signed-rank on equivalence-filtered kill ratios (paired by task); Cliff's δ. Surviving LLM mutants taxonomised by fault type; results feed RQ1b.
RQ1b: Benchmark Adequacy. dequate are benchmark test suites at detecting LLM-specific vs. traditional faults?	**H1:** suites show significantly lower kill rates against LLM-specific faults (from RQ1a) than against traditional mutants. **H0:** suites are equally adequate against both fault populations.	RQ1a equivalence-filtered mutant populations run against HumanEval and RealClassEval test suites. Metric: equivalence-filtered kill ratio per population.	Wilcoxon signed-rank (paired by task); Cliff's δ. Kill ratios correlated with cyclomatic complexity to test whether task difficulty moderates oracle weakness.
RQ2: Ranking Instability. ensitive are model rankings to semantically invariant paraphrases?	**H1:** pt variations cause rank inversions ($\tau_b < 0.85$) across model families on the intact benchmark. **H0:** ings remain stable across equivalent prompt variants.	100 paraphrases/task (fact reordering, persona-shifting, syntactic expansion) $\times$ 6 model families (e.g. Llama-3.1, Qwen2.5-Coder). Measured on the intact benchmark; re-applied per degradation level in RQ4.	Prompt templates decomposed into syntax trees; metamorphic relations applied. Kendall's τ_b + bootstrap 95 % CIs per model family. Per-transformation instability profile recorded.
RQ3: Metamorphic Validation. ehavioral divergence and the SBC Score detect false positives without human oracles?	**H1:** pairwise output disagreement across paraphrase-derived solutions predicts functional incorrectness even when benchmark tests pass. **H0:** ut consistency does not correlate with functional correctness.	k solutions per task (from k paraphrases) executed on M=200 shared random inputs. Cluster flagged *likely-incorrect* when disagreement rate $> \tau_{\text{div}}$ (calibrated in Tier 1 pilot). Consistency Assertions passed to RQ4.	Primary: pairwise output disagreement rate. Secondary: AST edit distance (structural diversity). SBC Score (Semantic + BLEU + Completeness) via reverse generation. ROC-AUC vs. oracle labels.
RQ4: Deficiency Mitigation & Interaction. T-augmented benchmarks restore rankings under oracle incompleteness? Does degradation amplify prompt sensitivity?	**H1a:** istency Assertions from RQ3 achieve significant rank recovery toward the full-suite baseline. **H0a:** ugmentation cannot compensate for oracle incompleteness. **H1b:** ing instability (τ_b) increases monotonically with degradation degree (compounding effect). **H0b:** pt sensitivity is independent of test-suite completeness.	Degrade test suites at 20 %, 50 %, 80 %. *MT-augmented = remaining tests + Consistency Assertions* (solutions from k paraphrases must agree on shared random inputs). RQ2 τ_b protocol re-run at each degradation level.	Rank recovery: Spearman ρ and τ_b vs. full-suite baseline. Interaction (H1b): τ_b regressed on degradation %; monotonicity via Page's L trend test. Cross-RQ: whether MT augmentation also stabilises rankings under paraphrase.

References

[1] Cosmic-ray: Mutation testing for python. `https://github.com/sixty-north/cosmic-ray`. Accessed: 2026-02-24.

[2] Pynguin: Automated unit test generation for python. `https://github.com/se2p/pynguin`. Accessed: 2026-02-24.

[3] Ali Asgari, Milan de Koning, Pouria Derakhshanfar, and Annibale Panichella. Metamorphic testing of deep code models: A systematic literature review. *arXiv preprint arXiv:2507.22610*, 2025.

[4] J De Curtò and I De Zarzà. Metamorphic testing for semantic invariance in large language models. *IEEE Access*, 13:214772–214791, 2025.

[5] Ziyu Li and Donghwan Shin. Mutation-based consistency testing for evaluating the code understanding capability of llms. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*, pages 150–159, 2024.

[6] Xiangyue Liu, Xiaobing Sun, Lili Bo, Yufei Hu, Xinwei Liu, and Zhenlei Ye. Evaluating the test adequacy of benchmarks for llms on code generation. *Journal of Software: Evolution and Process*, 37(7):e70034, 2025.

[7] Zhiyuan Pan, Xing Hu, Xin Xia, and Xiaohu Yang. Re-evaluating code llm benchmarks under semantic mutation. *arXiv preprint arXiv:2506.17369*, 2025.

[8] Ahilan Ayyachamy Nadar Ponnusamy. Bridging llm-generated code and requirements: reverse generation technique and sbc metric for developer insights. *arXiv preprint arXiv:2502.07835*, 2025.

[9] Musfiqur Rahman, SayedHassan Khatoonabadi, and Emad Shihab. Beyond synthetic benchmarks: Evaluating llm performance on real-world class-level code generation, 2025.

[10] Frank Tip, Jonathan Bell, and Max Schäfer. Llmorpheus: Mutation testing using large language models. *IEEE Transactions on Software Engineering*, 2025.